



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

**AgroSkopos: Desarrollo de
una plataforma web integral
para la telemetría y control de
un robot agrícola autónomo**



Presentado por Andrés Vera Cacho
en Universidad de Burgos — 8 de julio de 2026
Tutor: Rubén Ruiz González y Antonia Maiara
Marques do Nascimento



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



D. Rubén Ruiz González, profesor del departamento de Ingeniería Electromecánica, área de Ingeniería de Sistemas y Automática.

Expone:

Que el alumno D. Andrés Vera Cacho, con DNI 72903913P, ha realizado el Trabajo final de Grado en Ingeniería Informática titulado AgroSkopos.

Y que dicho trabajo ha sido realizado por el alumno bajo la dirección del que suscribe, en virtud de lo cual se autoriza su presentación y defensa.

En Burgos, 8 de julio de 2026

Vº. Bº. del Tutor:

Vº. Bº. del co-tutor:

D. Rubén Ruiz González

Dña. Antonia Maiara Marques do
Nascimento

Resumen

El presente Trabajo de Fin de Grado describe el diseño y desarrollo de AgroSkopos, una aplicación web integral destinada a la monitorización, telemetría y control remoto de un vehículo agrícola autónomo. Ante la necesidad actual de modernizar las explotaciones agrarias mediante herramientas de agricultura de precisión, este proyecto propone una solución centralizada basada en una arquitectura cliente-servidor de baja latencia.

El sistema centraliza las operaciones críticas del vehículo ofreciendo un mapa interactivo para el seguimiento de rutas, herramientas para la gestión del historial de misiones y un panel de control manual con soporte para transmisión de vídeo en vivo. Para validar la fiabilidad de la plataforma como paso previo a su integración con *hardware* físico, se ha implementado un motor de simulación nativo en el servidor. Este simulador emula el comportamiento cinemático y energético del robot, garantizando la correcta recepción y procesamiento en tiempo real de variables críticas como las coordenadas GPS, la velocidad y el nivel de batería.

A nivel técnico, la plataforma ha sido construida utilizando tecnologías modernas y modulares, destacando el uso de React y Zustand en el *frontend* para una gestión eficiente del estado, y Node.js junto con PostgreSQL en el *backend* para la lógica de negocio y persistencia de datos. Todo el ecosistema ha sido contenerizado mediante Docker, asegurando un despliegue robusto, escalable y fácilmente adaptable a futuros entornos de producción.

Descriptores

Telemetría, Agricultura de precisión, Robótica agrícola, Aplicación web, Tiempo real, Simulación, React, Node.js.

Abstract

This Final Degree Project describes the design and development of AgroSkopos, a comprehensive web application aimed at the monitoring, telemetry, and remote control of an autonomous agricultural vehicle. Given the current need to modernize farming operations through precision agriculture tools, this project proposes a centralized solution based on a low-latency client-server architecture.

The system centralizes the vehicle's critical operations by offering an interactive map for route tracking, tools for mission history management, and a manual control panel with support for live video streaming. To validate the platform's reliability as a preliminary step prior to its integration with physical *hardware*, a native simulation engine has been implemented on the server. This simulator emulates the kinematic and energetic behavior of the robot, ensuring the correct real-time reception and processing of critical variables such as GPS coordinates, speed, and battery level.

On a technical level, the platform has been built using modern and modular technologies, highlighting the use of React and Zustand on the *frontend* for efficient state management, and Node.js alongside PostgreSQL on the *backend* for business logic and data persistence. The entire ecosystem has been containerized using Docker, ensuring a robust, scalable deployment that is easily adaptable to future production environments.

Keywords

Telemetry, Precision agriculture, Agricultural robotics, Web application, Real-time, Simulation, React, Node.js.

Índice general

Índice general	iii
Índice de figuras	v
Índice de tablas	vi
Introducción	1
Objetivos del proyecto	3
2.1. Objetivo General	3
2.2. Objetivos Específicos	3
Conceptos teóricos	7
3.1. Agricultura de Precisión	7
3.2. Telemetría y Comunicaciones	9
3.3. Simulación en Robótica	9
Técnicas y herramientas	11
4.1. Metodologías y técnicas de gestión	11
4.2. Arquitectura de la aplicación	13
4.3. Tecnologías y <i>frameworks</i> principales	17
4.4. Gestión de código y entorno de desarrollo	23
Aspectos relevantes del desarrollo del proyecto	25
5.1. Motivación del proyecto	25
5.2. Inicio del proyecto y evolución del alcance	26
5.3. Adquisición de conceptos técnicos	26
5.4. Planificación	28

5.5. Desarrollo del Backend	29
5.6. Desarrollo del Frontend	32
5.7. <i>Testing</i> , calidad de código y despliegue	34
Trabajos relacionados	39
6.1. Soluciones Comerciales	39
6.2. Comparativa Funcional	40
Conclusiones y líneas de trabajo futuras	43
7.1. Conclusiones	43
7.2. Líneas de trabajo futuras	44
Bibliografía	47

Índice de figuras

3.1. Esquema de los macronutrientes primarios (NPK) y su función fundamental en el desarrollo vegetal	8
4.1. Tablero de <i>sprints</i> y gestión de incidencias en Zube	13
4.2. Diagrama de la arquitectura software	14
4.3. Diagrama de la arquitectura en capas del servidor Node.js	15
4.4. Diagrama de secuencia de la Arquitectura Orientada a Eventos por WebSockets	17
5.1. Flujo de Autenticación mediante JWT y Cookies HttpOnly	30
5.2. Diagrama ER inicial (SQL Crudo)	31
5.3. Diagrama ER autogenerado por Prisma	31
5.4. Panel principal de control (Dashboard) del sistema AgroSkopos . .	32
5.5. Gráfica comparativa de evolución temporal generada de forma nativa por la aplicación	33
5.6. Máquina de estados del ciclo de vida de una Misión	34
5.7. Panel principal de SonarCloud mostrando el estado del Quality Gate y la cobertura del proyecto	36

Índice de tablas

4.1. Resumen de herramientas y tecnologías por capa de arquitectura.	22
6.1. Comparativa de características entre plataformas	40

Introducción

La aplicación cubre los procesos esenciales de operación: desde el **control de misiones** y la **visualización telemétrica**, hasta la supervisión en vivo mediante el flujo de cámara y el control manual del vehículo. Al integrar estas funcionalidades en una plataforma modular, se pretende no solo agilizar los flujos de trabajo, sino ofrecer una herramienta que se adapte con facilidad a las exigencias de la agricultura de precisión.

Un pilar diferenciador de este proyecto es su **arquitectura orientada a la baja latencia** y el procesamiento dinámico de datos. Aunque el sistema fue concebido originalmente para su integración con una plataforma robótica física desarrollada en colaboración con el área de ingeniería electrónica, la solución actual emplea un robusto **simulador de telemetría nativo**. Este componente permite validar la lógica de control y monitorizar variables críticas (como el nivel de batería, las coordenadas GPS y el estado de carga), garantizando la seguridad operativa y la fiabilidad del *software* como paso previo a una futura implementación en *hardware* real.

Como parte de este Trabajo de Fin de Grado, se entregan diversos **materiales complementarios** que permiten evaluar tanto el código fuente como la plataforma en funcionamiento. El control de versiones y el **código completo del proyecto** se encuentran centralizados en el repositorio público de *GitHub*:

<https://github.com/andresveracachoo/aplicacion-robot-agricola>

Asimismo, la plataforma cuenta con un **despliegue operativo en la nube** alojando el *backend* en la infraestructura de Azure y el *frontend* en Vercel, accesible mediante el siguiente enlace:

<https://agroskopos.vercel.app/>

Para garantizar la robustez y mantenibilidad del *software*, se ha integrado un **análisis estático de código** realizado por SonarCloud [40], cuyos reportes de **calidad técnica** pueden visualizarse en:

https://sonarcloud.io/project/overview?id=AndresVeraCachoo_aplicacion-robot-agricola

Objetivos del proyecto

Este capítulo tiene como propósito definir el alcance y las metas fundamentales que guían el desarrollo de AgroSkopos. El proyecto surge de la necesidad de proporcionar una **interfaz de usuario avanzada** para la gestión de un vehículo autónomo en entornos agrícolas, priorizando la **precisión en la telemetría** y la **fiabilidad en el control remoto**.

2.1. Objetivo General

El objetivo principal de este Trabajo de Fin de Grado es el **diseño, implementación y validación** de una **plataforma web centralizada** para la monitorización, telemetría y control de un robot agrícola autónomo. La solución debe actuar como un **visor de precisión**, permitiendo al operador supervisar el estado del vehículo en tiempo real y gestionar misiones de trabajo de manera eficiente, garantizando la integridad operativa mediante un **entorno de simulación**.

2.2. Objetivos Específicos

Para alcanzar el objetivo general propuesto, se han definido una serie de metas específicas divididas en tres vertientes: funcionales, técnicas y de calidad y gestión.

Objetivos Funcionales

- **Visualización Geográfica:** Integrar un sistema de mapas interactivos basado en Leaflet [1] que permita el seguimiento preciso de la ubicación del robot y la representación de rutas de misión.
- **Panel de Telemetría:** Desarrollar un panel dinámico que procese y muestre variables críticas de estado, tales como el nivel de batería, la velocidad de avance y las coordenadas GPS.
- **Control Remoto y Vídeo:** Implementar una interfaz de control manual para la operación directa del vehículo, integrando un flujo de vídeo en baja latencia para la supervisión visual.
- **Gestión de Misiones:** Crear un sistema de persistencia para el historial de misiones, permitiendo la consulta de datos históricos y el análisis de rendimiento de jornadas anteriores.

Objetivos Técnicos

- **Arquitectura de Baja Latencia:** Configurar un sistema de comunicación cliente-servidor en tiempo real mediante canales bidireccionales (*WebSockets*) para asegurar que la telemetría refleje de forma instantánea el estado del vehículo y permitir el control remoto sin retardos.
- **Gestión de Estado Global:** Utilizar Zustand [32] para la administración del estado de la aplicación en el *frontend*, garantizando una reactividad fluida sin las sobrecargas de rendimiento asociadas a otras librerías.
- **Simulación Cinemática:** Desarrollar un motor de simulación nativo en el *backend* que emule el comportamiento físico y energético del robot, permitiendo validar la lógica de la aplicación sin dependencia inicial del *hardware* físico.
- **Arquitectura Asíncrona y Colas de Mensajes:** Incorporar un sistema de colas basado en Redis [35] para procesar tareas pesadas en segundo plano (como el envío de correos electrónicos transaccionales y la gestión de *tickets* de soporte), descongestionando el hilo principal del servidor.
- **Contenerización y Despliegue en la Nube:** Orquestar todo el ecosistema de *software* (interfaz, lógica de servidor, proxy inverso y

base de datos) mediante Docker [10] para asegurar la portabilidad, y desplegar la arquitectura completa en una máquina virtual alojada en Microsoft Azure [23].

Objetivos de Gestión y Calidad

- **Metodología Ágil (Scrum):** Implementar un marco de trabajo basado en Scrum [38], estructurado en iteraciones o *sprints* de dos semanas. Este enfoque permite gestionar la variabilidad en la disponibilidad temporal y garantiza una validación continua de los requisitos.
- **Pruebas Automatizadas y de Carga:** Diseñar e implementar una estrategia integral de pruebas, abarcando pruebas unitarias (*Vitest* [47]), pruebas de integración (*End-to-End* con *Supertest* [19]) y pruebas de estrés para evaluar el rendimiento bajo alta concurrencia mediante *k6* [15].
- **Integración Continua (CI/CD) y Calidad de Código:** Integrar plataformas de automatización como *GitHub Actions* [14] para validar el código antes de su despliegue, y emplear análisis estático mediante SonarCloud [40] para garantizar una deuda técnica nula y el cumplimiento estricto de *Quality Gates*.

Conceptos teóricos

Este capítulo introduce los **fundamentos teóricos necesarios** para comprender el dominio en el que opera el proyecto AgroSkopos. Se abordarán disciplinas que exceden el ámbito puro de la ingeniería del *software*, adentrándose en la **agronomía, la robótica de campo y los paradigmas de comunicación de alta frecuencia**.

3.1. Agricultura de Precisión

La agricultura de precisión es un **paradigma de gestión agrícola** basado en la observación, la medición y la respuesta a la variabilidad inter e intra-campo de los cultivos. Su objetivo es **optimizar el retorno de la inversión y preservar los recursos** mediante el uso intensivo de **tecnologías de la información (TIC)** y sistemas de teledetección [50, 26].

Variables Agronómicas Críticas: NPK, Humedad y pH

El correcto desarrollo de cualquier cultivo depende del equilibrio de diversas **variables edafológicas**, destacando especialmente la disponibilidad de macronutrientes, la retención hídrica y la acidez del suelo [13, 7].

- **Índice NPK (Nitrógeno, Fósforo y Potasio):** Son los macronutrientes primarios. El nitrógeno es fundamental para el crecimiento foliar; el fósforo es crítico en la transferencia de energía y el desarrollo radicular; y el potasio mejora la resistencia al estrés hídrico.
- **Humedad Volumétrica del Suelo:** Representa la cantidad de agua disponible en el perfil del suelo. Su monitorización evita tanto el estrés

por sequía como la asfixia radicular por encharcamiento, optimizando las estrategias de riego.

- **Potencial de Hidrógeno (pH):** Mide la acidez o alcalinidad del terreno. Es un factor determinante, ya que un pH fuera del rango óptimo (generalmente entre 6.0 y 7.0) "bloquea" los nutrientes, impidiendo que las raíces puedan absorber el NPK aunque este se encuentre presente.

La monitorización en tiempo real de estas variables permite aplicar dosis de fertilización e irrigación exclusivamente donde son necesarias, reduciendo la contaminación de acuíferos y minimizando los costes operativos.



Figura 3.1: Esquema de los macronutrientes primarios (NPK) y su función fundamental en el desarrollo vegetal

Radiación Solar Incidente

La **radiación solar activa fotosintéticamente** (PAR, por sus siglas en inglés) es la fracción del espectro electromagnético (400-700 nm) que las plantas pueden utilizar para la fotosíntesis. En el contexto de un robot agrícola autónomo propulsado por energía solar, la medición de esta radiación adquiere una doble utilidad: por un lado, se recopila como dato agronómico vital para estimar la evapotranspiración del cultivo; por otro, actúa como variable principal para calcular la **curva de recarga energética** de los paneles fotovoltaicos del propio vehículo.

3.2. Telemetría y Comunicaciones

La telemetría es una tecnología que permite la medición remota de magnitudes físicas y el posterior envío de la información hacia un operador del sistema [11].

Baja Latencia y Asincronía

En robótica móvil, la telemetría incluye datos críticos como la velocidad de avance, la orientación (guiñada, cabeceo, alabeo), las coordenadas satelitales (GNSS) y el nivel de energía. Dado que el operador requiere conocer el estado exacto del vehículo en el instante actual, la transmisión de estos datos no puede apoyarse en **protocolos de petición-respuesta convencionales**. En su lugar, se utilizan **WebSockets**, canales que permiten enviar ráfagas de datos asíncronos (eventos) con latencias que rondan los pocos milisegundos, garantizando la **seguridad en el control remoto** (telemando).

Sistemas de Colas de Mensajes

A diferencia de la telemetría, existen operaciones en un sistema integral (como el envío de notificaciones por correo electrónico o el procesamiento de *tickets* de soporte) que no requieren respuesta inmediata, pero sí garantía de entrega. Los **sistemas de colas asíncronas** almacenan estas tareas temporalmente en estructuras de datos en memoria (generalmente gestionadas por motores en memoria), liberando al servidor principal de cargas bloqueantes y distribuyendo el esfuerzo en trabajadores en segundo plano (*background workers*).

3.3. Simulación en Robótica

La integración de *hardware* y *software* en proyectos de robótica agrícola suele enfrentarse a bloqueos logísticos, problemas de suministro o fallos mecánicos. Para mitigar estos riesgos, la industria emplea la simulación en lazo cerrado (*Hardware/Software-In-The-Loop*).

Un **simulador cinemático** computa, a intervalos regulares de tiempo, el desplazamiento vectorial de un cuerpo en función de su velocidad actual, su rumbo y el terreno, generando coordenadas GPS virtuales. Si a esto se le suma un **simulador energético**, el sistema es capaz de drenar virtualmente la batería en función de la resistencia teórica del terreno y los motores, permitiendo a la capa de control (*software*) validar algoritmos críticos como

el Retorno al Punto de Origen (RTH, *Return To Home*) y sistemas de navegación autónoma [25] sin poner en riesgo una máquina física de alto coste.

Técnicas y herramientas

4.1. Metodologías y técnicas de gestión

Para el desarrollo del presente proyecto se ha prescindido de los modelos tradicionales de ciclo de vida del *software* (como el modelo en Cascada), optando en su lugar por un **enfoque ágil (*Agile*)**. Si bien esta decisión metodológica no fue la única responsable del éxito, su flexibilidad resultó vital para adaptarse a los imprevistos. Al realizar revisiones periódicas (aproximadamente cada dos semanas), fue posible realizar un **seguimiento progresivo** de la construcción del chasis físico del robot. Gracias a este enfoque iterativo, el hecho de que el *hardware* no llegara a construirse a tiempo no supuso un bloqueo repentino, sino que permitió **pivotar orgánicamente** hacia la creación de un simulador telemétrico sin llegar a paralizar el desarrollo del *software* en ningún momento.

Para materializar este enfoque, se ha aplicado una metodología pragmática que combina los principios iterativos de **Scrum** [38] con la gestión y trazabilidad de requisitos mediante **GitHub Issues**.

Scrum y adaptación al entorno académico

Scrum [38] es un marco de trabajo ágil orientado a la gestión de proyectos complejos. Se basa en el **desarrollo iterativo e incremental**, donde el trabajo se estructura en ciclos de tiempo fijo denominados **sprints**. Durante cada *sprint*, el equipo se compromete a diseñar, implementar y probar un subconjunto de funcionalidades priorizadas, generando un incremento de *software* potencialmente entregable al final del ciclo.

Dada la naturaleza estrictamente individual de este Trabajo de Fin de Grado, el marco oficial de Scrum [38] requirió una adaptación metodológica para encajar en el contexto académico:

- ***Sprints* de dos semanas:** Se estableció una cadencia de desarrollo basada en iteraciones de catorce días. Esta ventana temporal proporcionó el equilibrio óptimo entre la obtención de resultados tangibles y la flexibilidad necesaria para absorber los picos de baja disponibilidad derivados del calendario académico y los periodos de exámenes.
- **Revisión de *Sprint* y validación de negocio:** Al finalizar cada iteración, se programaron reuniones de seguimiento con los tutores del proyecto. En este escenario, asumieron un rol análogo al de *Product Owner*, evaluando los incrementos funcionales de la aplicación, aportando retroalimentación técnica y validando que los requisitos cumplieran con los estándares de ingeniería exigidos.

De forma complementaria a los ciclos de Scrum, se adoptó *GitHub Issues* como herramienta principal para el seguimiento operativo diario y la documentación de tareas.

Cada incidencia (*issue*) fue documentada detallando su propósito técnico y asignándole etiquetas (*labels*) estandarizadas que indicaban su prioridad y naturaleza funcional (por ejemplo, *testing* para añadir tests o *bug* para la resolución de fallos). Al estar estrechamente ligado al control de versiones, este enfoque facilitó asociar de forma automática las confirmaciones de código (*commits*).

Finalmente, la sincronización bidireccional entre *GitHub Issues* y la plataforma ágil Zube [51] permitió agrupar estas incidencias para planificar los *sprints* de forma visual. Esta integración garantizó una trazabilidad absoluta del progreso, asegurando que el esfuerzo de programación estuviera siempre documentado y alineado con los objetivos de la iteración en curso.

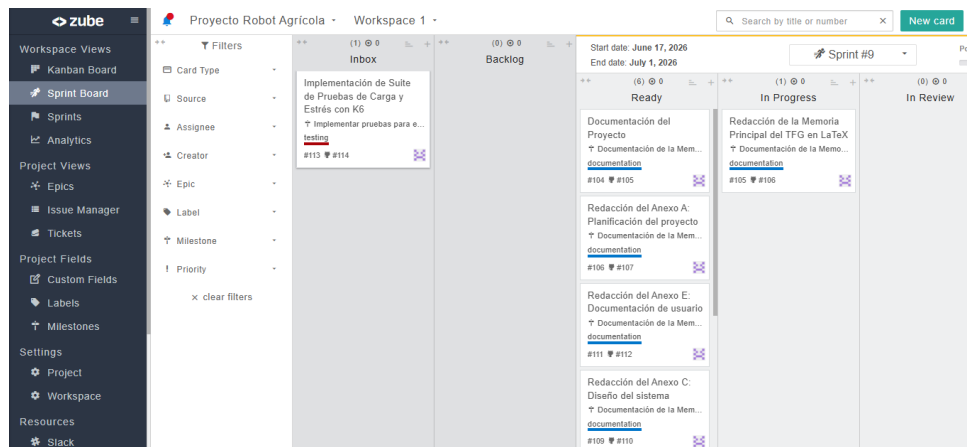


Figura 4.1: Tablero de *sprints* y gestión de incidencias en Zube

4.2. Arquitectura de la aplicación

El diseño del sistema AgroSkopos se ha concebido bajo premisas de alta cohesión, bajo acoplamiento y reactividad en tiempo real, requisitos indispensables para una plataforma de telemetría robótica. Para satisfacer estas necesidades, el *software* se fundamenta en la combinación de varios patrones arquitectónicos complementarios.

Arquitectura Cliente-Servidor y *Single Page Application* (SPA)

A nivel macroarquitectónico, el sistema implementa un modelo **Cliente-Servidor** con un **desacoplamiento estricto** entre la interfaz de usuario (*frontend*) y la lógica de negocio (*backend*). A pesar de esta separación operativa e independiente, el código de ambas partes (`/app/client` y `/app/server`) se administra de manera centralizada mediante el patrón de **Monorepo**, lo que optimiza la gestión de dependencias y simplifica los flujos de integración continua sin romper el aislamiento técnico de cada capa.

El cliente web ha sido desarrollado siguiendo el patrón de **Arquitectura de Una Sola Página** (*Single Page Application* o SPA). A diferencia de las aplicaciones web tradicionales multi-página, la SPA carga un único documento HTML en el navegador y actualiza dinámicamente el contenido (el Modelo de Objetos del Documento o DOM) a medida que el usuario interactúa con la plataforma. Este enfoque reduce drásticamente el consumo de ancho de banda y proporciona una **fluidez de navegación** similar a la

de una aplicación de escritorio, algo crítico al operar el mapa interactivo y el panel de control del robot.

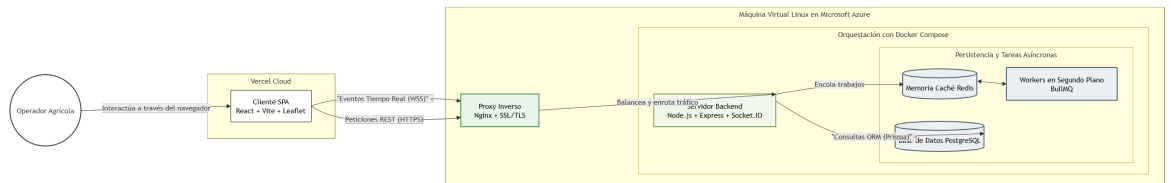


Figura 4.2: Diagrama de la arquitectura software

Arquitectura en Capas (*Layered Architecture*) en el *Backend*

Para garantizar la mantenibilidad y escalabilidad del servidor, el *backend* implementa una Arquitectura en Capas lógicas. El flujo de información desde que entra una petición hasta que se consulta la base de datos atraviesa niveles con responsabilidades únicas y bien definidas:

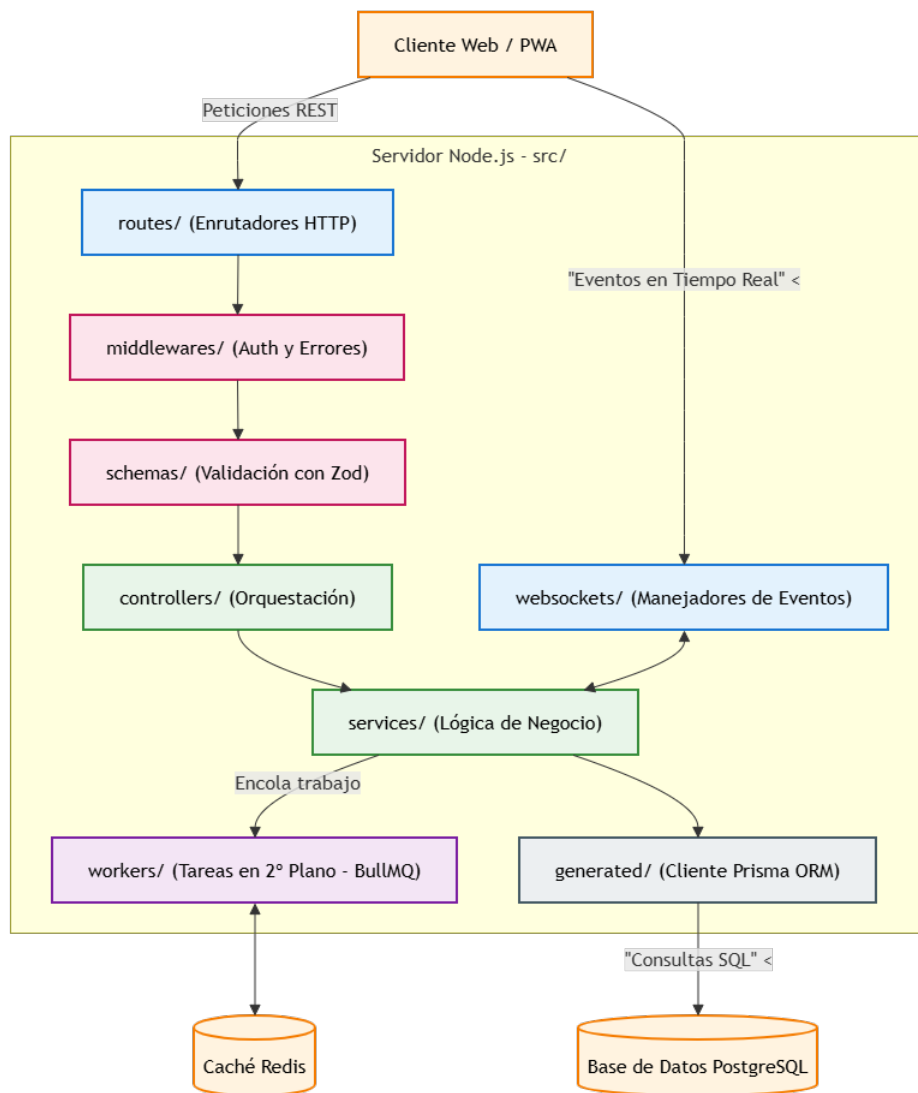


Figura 4.3: Diagrama de la arquitectura en capas del servidor Node.js

1. **Capa de Enrutamiento (*Routes*):** Actúa como punto de entrada de la API REST, interceptando las peticiones HTTP (GET, POST, PUT, DELETE) y derivándolas al controlador correspondiente.
2. **Capa de Intermediación (*Middlewares*):** Subsistema transversal encargado de interceptar el tráfico para aplicar reglas de seguridad (verificación de *tokens* de autenticación) y gestión centralizada de errores antes de alcanzar la lógica principal.

3. **Capa de Aplicación (*Controllers*):** Contiene la lógica orquestadora para procesar las peticiones HTTP, validando los datos de entrada y enviando la respuesta al cliente.
4. **Capa de Servicios (*Services*):** Contiene la lógica de negocio pura y la lógica asíncrona pesada (como el envío de correos o la integración con Redis [35] y BullMQ [41]). Abstrae la complejidad para mantener los controladores limpios y aislados.
5. **Capa de Persistencia (*Data/Config*):** Abstrae la conexión a la base de datos PostgreSQL [43] delegando el mapeo relacional a Prisma ORM, lo que garantiza consultas fuertemente tipadas y seguras.

Arquitectura Orientada a Eventos (*Event-Driven*)

Mientras que las operaciones de gestión (como consultar el historial de misiones o registrar usuarios) se resuelven mediante el clásico **patrón de petición-respuesta (API REST)**, la monitorización cinemática y energética del robot exige un paradigma distinto.

Para solventar esta necesidad, AgroSkopos integra una Arquitectura Orientada a Eventos paralela a la API REST. Mediante el uso de WebSockets bidireccionales, el sistema establece un canal de comunicación persistente y de baja latencia. El simulador del robot actúa como un "Productor" de eventos (emitiendo variables de estado como batería, velocidad y coordenadas cada pocos milisegundos), el servidor actúa como un "*Broker*" o despachador, y los componentes visuales del *frontend* actúan como Consumidores, reaccionando y re-renderizando la pantalla instantáneamente tras la recepción de un nuevo evento. Este patrón es el núcleo que permite que el visor funcione en estricto tiempo real.

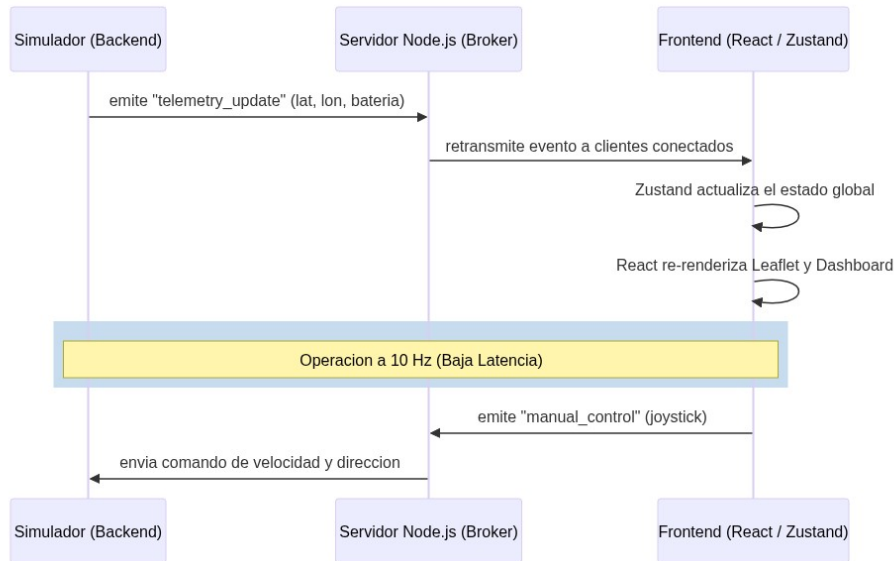


Figura 4.4: Diagrama de secuencia de la Arquitectura Orientada a Eventos por WebSockets

4.3. Tecnologías y *frameworks* principales

En esta sección se detallan las herramientas tecnológicas que conforman el ecosistema del proyecto, divididas según la capa arquitectónica en la que operan.

Backend (Lógica de servidor)

- **Node.js [31] y Express [30]:** Node.js [31] es un entorno de ejecución multiplataforma basado en el motor V8 de JavaScript. Sobre este entorno se emplea Express [30], un *framework* web minimalista que facilita la creación de la API REST.
- **PostgreSQL [43] y Prisma ORM [33]:** PostgreSQL [43] constituye el núcleo de persistencia del sistema relacional. Para la interacción con la base de datos se utiliza Prisma ORM [33], que sustituye a los clientes SQL tradicionales proporcionando un esquema declarativo, migraciones automatizadas y un cliente de base de datos fuertemente tipado en JavaScript, mitigando radicalmente el riesgo de inyecciones SQL.

- **Redis [35] y BullMQ [41]:** Redis [35] es un almacén de estructura de datos en memoria empleado como un intermediario (*broker*) de alta velocidad. En conjunción con BullMQ [41], permite la gestión robusta de colas de mensajes y trabajos en segundo plano (*background jobs*), siendo la espina dorsal del sistema asíncrono para el envío de correos electrónicos y la gestión de *tickets* de soporte técnico sin bloquear el flujo principal de peticiones de la API.
- **Socket.io [39] (Servidor):** Socket.io [39] actúa como el motor de la arquitectura orientada a eventos, permitiendo la transmisión en tiempo real de la telemetría del robot sin la sobrecarga de cabeceras de las peticiones HTTP tradicionales.
- **Seguridad de la API (JWT, Bcrypt, Helmet y Rate Limit):** Para garantizar la integridad y confidencialidad de la API, se ha implementado un ecosistema de seguridad compuesto por varias librerías:
 - **JSON Web Token [4] (jsonwebtoken):** Estándar abierto (RFC 7519) utilizado para transmitir información de forma segura entre el cliente y el servidor en formato JSON. Se emplea para la autenticación sin estado (*stateless*) tras el inicio de sesión.
 - **Bcrypt [6]:** Librería de cifrado basada en el algoritmo de *hash* de contraseñas de Blowfish. Garantiza que las credenciales de los usuarios se almacenen de forma segura y unidireccional.
 - **Helmet [16] y Express-Rate-Limit [27]:** *Middlewares* de seguridad que protegen la aplicación configurando adecuadamente las cabeceras HTTP y limitando el número máximo de peticiones por IP, mitigando ataques de denegación de servicio (DDoS) y ataques de fuerza bruta.
- **Zod [8]:** Biblioteca de validación de esquemas estáticos y declaración de tipos mediante un enfoque *schema-first*. Garantiza que las estructuras de datos que entran a la API cumplan con formatos estrictos antes de ser procesadas.
- **Nodemailer [2]:** Módulo para aplicaciones Node.js que permite el envío automatizado de correos electrónicos. Destaca por su seguridad (soporte para TLS/STARTTLS) e independencia del proveedor de correo.
- **Turf.js [44]:** Biblioteca avanzada de análisis espacial en JavaScript. Permite realizar operaciones matemáticas geoespaciales complejas de forma nativa (cálculo de áreas, intersecciones de polígonos y distancias).

Frontend (Interfaz de usuario)

- **React [22] y Vite [48]:** React [22] es una biblioteca de JavaScript desarrollada por Meta para la construcción de interfaces de usuario basadas en componentes declarativos y reutilizables. Su uso permite gestionar dinámicamente el DOM mediante un modelo de renderizado reactivo. Para la construcción y empaquetado del código se emplea Vite [48], una herramienta de desarrollo de nueva generación que aprovecha los módulos ES nativos del navegador para ofrecer un entorno de desarrollo ultrarrápido y compilaciones optimizadas.
- **React Router [36]:** React Router [36] es la biblioteca estándar de enrutamiento para aplicaciones React [22]. Habilita la navegación dinámica entre distintas vistas del panel de control (*Dashboard*, Mapa, Historial, Perfil) sin necesidad de recargar la página web, consolidando la arquitectura de *Single Page Application* (SPA).
- **Zustand [32]:** Zustand [32] es un gestor de estado global ligero, rápido y escalable. A diferencia de soluciones más pesadas como Redux, Zustand [32] utiliza *hooks* de React [22] para compartir el estado (como los datos telemétricos en vivo o el usuario activo) entre componentes profundamente anidados sin incurrir en problemas de renderizado innecesario ni excesivo código repetitivo (*prop-drilling*).
- **Ecosistema Leaflet [1] (Geoman y Heatmaps):** Leaflet [1] es la principal biblioteca de código abierto para la creación de mapas interactivos adaptables a dispositivos móviles. Para su integración reactiva se emplea React-Leaflet [1]. Este núcleo cartográfico se ha potenciado con dos extensiones fundamentales para el sector agrícola:
 - **Leaflet-Geoman [1]:** Herramienta que proporciona controles avanzados de dibujo y edición geométrica, permitiendo al usuario trazar polígonos (parcelas) y rutas directamente sobre el visor espacial.
 - **Leaflet.heat [1]:** *Plugin* para la generación de mapas de calor, utilizado para representar gráficamente la densidad de datos edafológicos (como humedad o nutrientes) sobre el terreno.
- **Axios [5]:** Cliente HTTP basado en promesas diseñado para gestionar la comunicación asíncrona entre aplicaciones web y APIs REST, facilitando la interceptación de peticiones y respuestas.

- **Recharts** [34]: Biblioteca de visualización de datos basada en componentes de React [22] y D3.js. Permite modelar información numérica mediante gráficos interactivos y personalizables.
- **html2canvas** [28] y **jsPDF** [18]: Conjunto de herramientas para la exportación de documentos. *html2canvas* [28] permite capturar el DOM del navegador y convertirlo en una imagen (lienzo), mientras que *jsPDF* [18] empaqueta dichas imágenes en documentos portátiles (PDF) estructurados.
- **date-fns** [37]: Biblioteca de utilidades para la manipulación y formateo de fechas en JavaScript. Ofrece un diseño modular que permite importar solo las funciones necesarias, reduciendo el tamaño del paquete final.
- **Internacionalización (i18next** [17]): *Framework* que dota al sistema de soporte multilingüe, adaptando el contenido de la interfaz en función de la configuración regional del usuario.
- **PWA (*Progressive Web App*)** [46]: Estándar de desarrollo que dota a las aplicaciones web de capacidades nativas (instalabilidad, ejecución en segundo plano y operación sin conexión). Mediante *Service Workers*, cachea los recursos estáticos para acelerar los tiempos de carga en redes inestables.

Testing y Calidad del *Software*

- **Jest** [21] y **Supertest** [19]: Jest [21] es un marco de pruebas unitarias para JavaScript diseñado para garantizar la correctitud del código. Se complementa con Supertest [19], una biblioteca de aserciones especializada en simular peticiones HTTP y probar APIs REST sin necesidad de levantar un servidor en un puerto real.
- **Testcontainers** [3]: Herramienta que soporta pruebas de integración mediante el despliegue programático de contenedores Docker [10] [10] efímeros (como bases de datos desechables) durante la ejecución de los tests.
- **Vitest** [47] y **React Testing Library** [42]: Vitest [47] es un marco de pruebas rápido y nativo impulsado por Vite [48]. Junto con React Testing Library [42], permite escribir pruebas de componentes de interfaz de usuario centrándose en el comportamiento y accesibilidad en lugar de en los detalles de implementación.

- **k6 [15]**: Herramienta de código abierto para pruebas de carga y rendimiento (*load testing*). Permite inyectar cientos de usuarios virtuales concurrentes para descubrir cuellos de botella y evaluar el comportamiento de los servidores bajo condiciones de estrés.

Infraestructura Cloud y Contenedores

- **Docker [10] [10] y Docker [10] Compose [10]**: Plataforma de contenerización que permite empaquetar aplicaciones junto con sus dependencias en unidades estándar aisladas. Docker [10] Compose facilita la orquestación de múltiples contenedores interconectados mediante un único archivo declarativo.
- **Vercel [45]**: Plataforma de *Cloud Computing* optimizada para el despliegue de aplicaciones web modernas. Destaca por su integración continua y el uso de una Red de Entrega de Contenido (CDN) distribuida globalmente.
- **Microsoft Azure [23]**: Proveedor de nube pública que ofrece Infraestructura como Servicio (IaaS), permitiendo el despliegue de máquinas virtuales escalables y seguras.
- **Nginx [12]**: Servidor web y proxy inverso de alto rendimiento. Actúa como barrera de seguridad perimetral, gestionando dinámicamente el enrutamiento HTTP/HTTPS y el balanceo de carga.

Integración Continua y Análisis Estático

- **GitHub Actions [14]**: Plataforma de automatización que permite definir flujos de trabajo (*workflows*) directamente en el repositorio. Actúa como el motor de la Integración Continua (CI), ejecutando pruebas y analizando el código automáticamente tras cada confirmación.
- **SonarCloud [40]**: Servicio basado en la nube para el análisis estático de código. Evalúa la calidad del *software* midiendo variables como la deuda técnica, las vulnerabilidades de seguridad y el porcentaje de cobertura de pruebas (*Coverage*), bloqueando el despliegue si no se superan los umbrales de calidad exigidos (*Quality Gates*).

Herramienta / Tecnología	<i>Frontend</i>	<i>Backend</i>	Infraestructura
React / Vite	X		
Zustand	X		
Leaflet (Geoman, Heat)	X		
Axios / Recharts	X		
Node.js / Express		X	
PostgreSQL / Prisma ORM		X	
Redis / BullMQ		X	
Socket.io	X	X	
JWT / Bcrypt / Zod		X	
Nodemailer		X	
Jest / Supertest		X	
Vitest / React Testing Library	X		
k6		X	
Docker [10] / Docker [10] Compose			X
Vercel			X
Microsoft Azure / Nginx			X
GitHub Actions / SonarCloud			X

Tabla 4.1: Resumen de herramientas y tecnologías por capa de arquitectura.

Documentación y diseño

Para la redacción, estructuración y composición tipográfica de la presente memoria técnica se ha utilizado inicialmente el sistema $\text{\LaTeX}$ [20] a través de la plataforma web Overleaf [9]. La elección de estas herramientas responde a su excelente rendimiento en el manejo de documentos de carácter científico-técnico y a su capacidad para garantizar un formato limpio, uniforme y profesional. En una primera fase, Overleaf facilitó la automatización de elementos complejos como los índices dinámicos, las referencias cruzadas, el listado de figuras y la gestión bibliográfica.

A pesar de tratarse de un desarrollo de *software* individual, la plataforma web resultó fundamental para optimizar el flujo de trabajo inicial con los tutores del proyecto, habilitando un acceso compartido para la supervisión asíncrona de los borradores en tiempo real.

No obstante, en las etapas más avanzadas de la redacción, y por una estricta cuestión de comodidad y centralización del entorno de trabajo, se procedió a descargar e instalar una distribución de $\text{\LaTeX}$ de forma local en el equipo de desarrollo. A partir de este momento, el grueso de la documentación

continuó redactándose y compilándose directamente desde **Visual Studio Code** [24] [24], aprovechando la potente extensión **LaTeX Workshop** [49]. Este cambio permitió integrar el flujo de escritura de la memoria con el mismo editor utilizado para el desarrollo del código fuente, ganando agilidad y unificando todo el proyecto en un único entorno local.

4.4. Gestión de código y entorno de desarrollo

Control de versiones y flujo de trabajo individual

El control de versiones del proyecto se ha gestionado mediante Git, empleando GitHub como repositorio remoto centralizado y plataforma de respaldo para el código fuente. Al tratarse de un desarrollo de *software* individual y no colaborativo, se descartó el uso de modelos de ramificación complejos (como GitFlow), los cuales añaden una sobrecarga innecesaria de fusión de ramas (*merging*) y resolución de conflictos.

En su lugar, se adoptó un flujo de trabajo simplificado enfocado a la trazabilidad y la estabilidad del código. El desarrollo se centralizó directamente sobre la rama principal o mediante ramas efímeras de corta duración vinculadas a tareas concretas. Este enfoque lineal garantizó que el historial de desarrollo permaneciera limpio y comprensible, asegurando que cada incremento de funcionalidad fuera registrado cronológicamente sin comprometer la velocidad de despliegue ni la integridad de la aplicación.

Conventional Commits Para mantener la disciplina y el rigor metodológico en el repositorio, la confirmación de cambios se ha regido por la especificación semántica de *Conventional Commits*. Cada mensaje de registro (*commit*) se ha estructurado utilizando prefijos obligatorios que identifican unívocamente la naturaleza de la modificación, tales como "feat" para nuevas características, "fix" para correcciones de errores o "test" para la infraestructura de pruebas. Esta práctica ha dotado al proyecto de una legibilidad excelente, facilitando la auditoría del *software* y la generación de un historial de cambios profesional.

Entorno operativo y administración

El Entorno de Desarrollo Integrado (IDE) principal escogido para la codificación de toda la plataforma ha sido Visual Studio Code [24], debido a su ligereza, naturaleza de código abierto y su soporte nativo para el

ecosistema de JavaScript y Node.js [31]. Para optimizar el ciclo de desarrollo y asegurar la homogeneidad del *software*, la potencia del editor se ha ampliado mediante un conjunto seleccionado de extensiones técnicas:

- **ESLint [29] y Prettier:** Encargados de realizar el análisis estático y el formateo en tiempo de ejecución dentro del editor. Forzaron el cumplimiento de las reglas de estilo y detectaron errores de sintaxis o variables huérfanas antes de la compilación.
- **Docker [10] y Containers (Microsoft):** Extensiones para la inspección visual, gestión y monitorización en tiempo real de los contenedores locales (*Frontend*, *Backend* y Base de Datos), agilizando las pruebas de despliegue y la orquestación del entorno.
- **SonarLint:** Integrado como un *linter* avanzado en el IDE, conectándose de forma bidireccional con las reglas de calidad de SonarCloud [40] para detectar vulnerabilidades de seguridad (*Security Hotspots*) y deuda técnica en el mismo instante de la escritura del código.
- **Database Client for PostgreSQL [43]:** Herramienta integrada en el propio editor que permitió la conexión directa a la base de datos local y a los contenedores, facilitando la ejecución de sentencias SQL y la validación del esquema de datos sin necesidad de recurrir a clientes pesados externos.
- **LaTeX Workshop [49]:** Extensión clave que posibilitó la compilación, previsualización en tiempo real y detección de errores de sintaxis al redactar toda la memoria técnica y los anexos del proyecto directamente desde el IDE.

Aspectos relevantes del desarrollo del proyecto

En este capítulo se describe el ciclo de vida completo del proyecto AgroS-kopos, abordando desde su concepción inicial y planificación metodológica, hasta las fases de implementación técnica y validación.

5.1. Motivación del proyecto

El origen de este Trabajo de Fin de Grado surge del interés por aplicar y consolidar los conocimientos adquiridos en la titulación en un entorno práctico multidisciplinar. La motivación principal fue el diseño de una plataforma de telemetría y control agrícola capaz de integrarse de forma colaborativa con otro Trabajo de Fin de Grado perteneciente a la rama de Ingeniería Electrónica. Mientras que dicho proyecto abordaba el diseño, ensamblaje y la programación a bajo nivel del chasis físico de un robot agrícola, el presente trabajo asumió el reto de desarrollar de forma íntegra toda la capa de *software* de alto nivel.

Esto incluía la creación de la infraestructura de comunicaciones, la persistencia de datos y la interfaz de usuario que permitiría gobernar dicho vehículo de forma remota. Esta propuesta representó un desafío técnico sumamente enriquecedor, al permitir la convergencia metodológica entre el desarrollo de aplicaciones web modernas orientadas a la baja latencia y la interacción directa con el *hardware* en el ámbito del Internet de las Cosas (IoT).

5.2. Inicio del proyecto y evolución del alcance

En la fase inicial de toma de requisitos, planteada a principio de septiembre, el alcance del proyecto era considerablemente más reducido y lineal. El objetivo original consistía en desarrollar una aplicación funcional que cubriera la gestión básica de usuarios y permitiría visualizar la telemetría del robot en tiempo real. En cuanto al control, únicamente se contemplaba una navegación rudimentaria mediante el envío de coordenadas individuales basadas en los clics del operador sobre un mapa, complementado con tablas y gráficas estáticas para el análisis de los datos.

Sin embargo, durante el transcurso del curso académico, el proyecto experimentó una reestructuración forzosa. Diversos problemas técnicos, logísticos y de suministro impidieron que el proyecto de Ingeniería Electrónica pudiera finalizar la construcción del robot físico a tiempo. Esta eventualidad imposibilitó la fase de integración de *hardware* que estaba prevista inicialmente.

Lejos de suponer un bloqueo, este contratiempo impulsó una ambiciosa evolución orgánica del *software*. Para poder validar la arquitectura orientada a eventos y la recepción de telemetría, el **motor de simulación interno** (que inicialmente se había concebido como un simple *script* de pruebas temporales) se reescribió por completo y se dotó de algoritmos cinemáticos avanzados. Al no requerir la inversión de tiempo en la integración física, el esfuerzo de desarrollo se redirigió hacia la ampliación masiva de las funcionalidades de la plataforma, convirtiendo lo que iba a ser un simple visor de datos en un producto tecnológico integral mucho más completo.

De este modo, se incorporó un complejo sistema de **misiones autónomas y planificación de rutas**, delimitación virtual del terreno (geocercas), soporte multilenguaje (i18n), personalización de la interfaz (modo oscuro) y la migración arquitectónica hacia una **Aplicación Web Progresiva (PWA)**.

5.3. Adquisición de conceptos técnicos

El desarrollo de una aplicación con requerimientos de telemetría, simulación cinemática y mapas interactivos en tiempo real supuso un desafío técnico significativo, exigiendo un periodo intensivo de capacitación tecnológica previo a la fase de codificación. Puesto que el *stack* tecnológico

empleado (PERN: PostgreSQL [43], Express [30], React [22] y Node.js [31]) difería notablemente de las tecnologías abordadas con profundidad a lo largo del plan de estudios del grado, el proyecto requirió un firme proceso de investigación y autoaprendizaje.

Esta adquisición de competencias se llevó a cabo en dos etapas diferenciadas:

- **Etapla Inicial (Verano 2025):** El proceso de formación comenzó durante el periodo estival, coincidiendo con las prácticas extracurriculares en empresa. En esta fase se llevó a cabo un estudio autodidacta centrado en los fundamentos del desarrollo web moderno, asimilando la sintaxis y capacidades de JavaScript (ES6+), así como la estructuración y diseño con HTML5 y CSS3. Esta etapa sentó las bases necesarias para abordar la programación orientada a la web con la agilidad requerida.
- **Etapla de Pre-desarrollo (Septiembre - Octubre 2025):** Tras la formalización de los requisitos del proyecto, se estableció un bloque de un mes dedicado de forma exclusiva a la investigación teórica, antes de realizar la primera confirmación de código (*commit*). Durante este tiempo se consolidó el dominio del ecosistema de React [22] y de las tecnologías fundamentales para AgroSkopos:
 - **Paradigma de React y Estado Global:** Se profundizó en el modelo declarativo, el ciclo de vida de los componentes y la gestión avanzada de estados globales mediante *hooks* optimizados como Zustand [32], previendo el alto volumen de datos de telemetría que la aplicación debía manejar.
 - **Librerías Geoespaciales (Leaflet):** Sabiendo que el núcleo visual del proyecto iba a ser un visor interactivo de precisión, se realizó un estudio pormenorizado de la API de Leaflet [1]. Se analizaron conceptos cartográficos como las capas de teselas (*tile layers*), las proyecciones de coordenadas y la renderización reactiva de polígonos y marcadores.

Esta fase preparatoria resultó crítica para mitigar los riesgos técnicos del proyecto. El tiempo invertido garantizó que, una vez iniciado el desarrollo *frontend*, se ejecutara bajo decisiones de diseño de *software* maduras, limpias y escalables, evitando la deuda técnica temprana.

5.4. Planificación

La planificación del proyecto se estructuró asumiendo la alta incertidumbre derivada de un desarrollo dependiente, en sus inicios, de factores externos (el *hardware*). Como se documentó en el capítulo metodológico, se empleó **Scrum** junto a la herramienta **Zube** para orquestar el trabajo en iteraciones o *sprints* de dos semanas.

Esta fragmentación en ciclos de entrega cortos permitió aislar la complejidad y otorgó una gran capacidad de adaptación ante los cambios de alcance. De este modo, se garantizó que el núcleo de la aplicación estuviera siempre consolidado y operativo antes de implementar nuevas capas de lógica de negocio o complejas mecánicas de visualización por encima de él.

El ciclo de desarrollo se dividió transversalmente en las siguientes grandes fases o hitos planificados:

1. **Fase de Análisis y Diseño:** Toma de requisitos iniciales, diseño de la base de datos relacional y elaboración de los prototipos de la interfaz gráfica de usuario.
2. **Fase de Desarrollo *Frontend*:** Construcción de la *Single Page Application* (SPA) con React [22], integración del mapa interactivo con Leaflet [1] y conexión con los servicios del *backend*.
3. **Fase de Desarrollo *Backend*:** Implementación del servidor Node.js [31], despliegue del contenedor PostgreSQL [43], configuración de la API REST y establecimiento del servidor de WebSockets para la comunicación bidireccional.
4. **Fase de Refactorización y Simulación Avanzada:** Hito introducido tras la cancelación de las pruebas físicas. Consistió en el desarrollo del motor de simulación cinemática y la generación dinámica de datos agronómicos para replicar el comportamiento de un entorno real.
5. **Fase de Expansión de Funcionalidades:** Implementación de las misiones, algoritmos de navegación (rutas de cobertura y *zigzag*), internacionalización y configuración de la PWA.
6. **Fase de Pruebas (*Testing*) y Aseguramiento de la Calidad:** Consolidación de las pruebas unitarias y E2E, y análisis estático con SonarCloud [40].

7. **Fase de Pruebas de Carga y Despliegue en la Nube:** Pruebas de carga con k6 [15], contenedorización final mediante Docker [10] Compose y orquestación productiva en Vercel [45] y Azure [23].

5.5. Desarrollo del Backend

La implementación de la lógica del servidor de AgroSkopos fue un proceso iterativo que evolucionó conforme a las crecientes necesidades del sistema, priorizando siempre la estabilidad y la escalabilidad.

Arquitectura, Seguridad y Modelado de Datos inicial

Una vez cimentadas las bases visuales en las primeras semanas de trabajo, el enfoque se trasladó al servidor **Node.js** [31]. Se abandonaron los datos estáticos migrando el núcleo de almacenamiento a **PostgreSQL** [43]. En esta fase temprana, el diseño de la persistencia de datos persiguió una filosofía de extrema simplicidad para evitar la sobreingeniería inicial. Originalmente, el sistema operaba únicamente con tres tablas maestras: *Usuarios* (gestión de acceso), *Misiones* (estado de los recorridos del robot) y *Telemetría* (captura de datos masivos). La comunicación con la base de datos se realizaba mediante consultas SQL crudas integradas directamente en el código del servidor.

Paralelamente, se implementaron los pilares de seguridad del sistema. Se integró el cifrado asimétrico de contraseñas mediante **Bcrypt** y un robusto sistema de autenticación basado en **JSON Web Tokens (JWT)**, complementado con una lógica de Control de Acceso Basado en Roles (RBAC) para proteger las rutas de la API de usuarios no autorizados.

Para mitigar vulnerabilidades críticas como los ataques de secuestro de sesión (*Cross-Site Scripting* o XSS), se diseñó un mecanismo de doble *token* (*AccessToken* de vida corta y *RefreshToken* de vida larga). Estos *tokens* no se inyectan en el *LocalStorage* del navegador (que es vulnerable a la inyección de *scripts*), sino que viajan directamente en **Cookies HttpOnly**. Esto asegura que las credenciales sean totalmente inaccesibles desde el código del cliente y se adjunten de forma automática y transparente en cada petición.

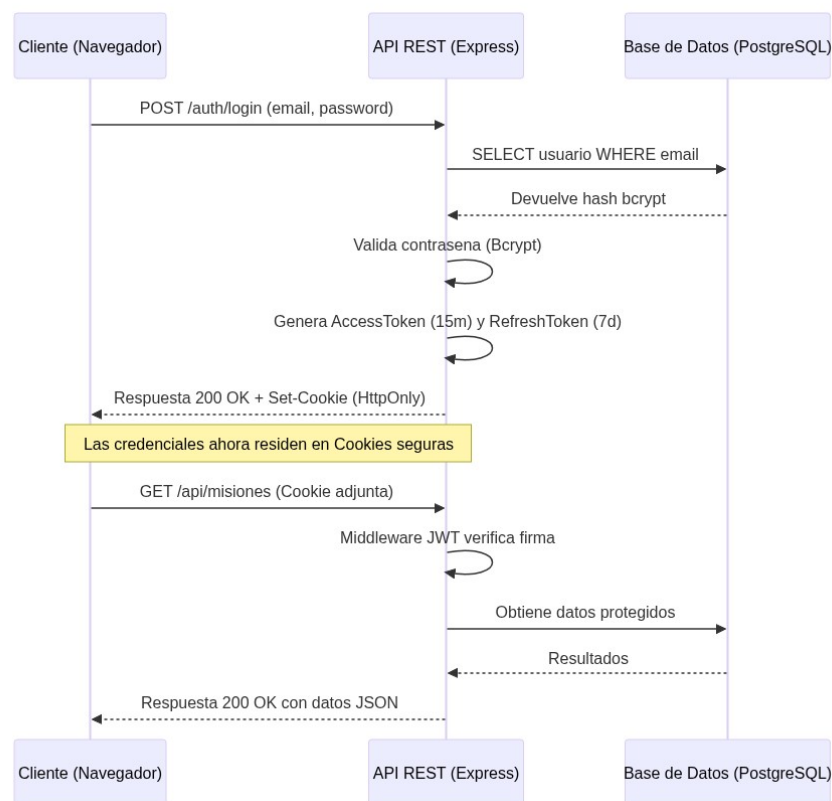


Figura 5.1: Flujo de Autenticación mediante JWT y Cookies HttpOnly

Evolución del ORM y Validación de Datos

A medida que creció la complejidad de las funcionalidades y aumentaron las tablas y relaciones de la aplicación, se hizo evidente la necesidad de escalar el sistema. Para ello, se integró el motor de persistencia **Prisma ORM**. Prisma permitió mantener una estructura de datos limpia pero abstraigo por completo la redacción manual de SQL, mitigando el riesgo de inyección de código y delegando al *framework* la generación automática de migraciones.

Aún más importante, Prisma actuó como el pilar fundamental para lograr un **Tipado Estricto de Extremo a Extremo** (*End-to-End Type Safety*). Al combinar los tipos autogenerados por la base de datos de Prisma con la librería de validación estática **Zod** en la entrada de las peticiones HTTP, se garantizó que la información fluyera con una integridad estructural matemática, detectando discrepancias mucho antes del tiempo de ejecución.

Además, para unificar las respuestas de fallo (como un *payload* malformado de Zod o un fallo interno del servidor) y evitar que se filtren trazas de pila sensibles en producción, se diseñó un **Middleware de Manejo Global de Errores** que actúa como sumidero final y devuelve errores HTTP estandarizados y limpios al cliente.

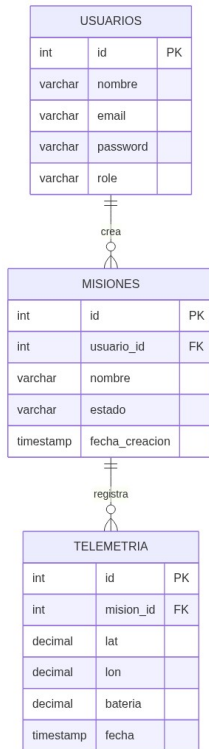


Figura 5.2: Diagrama ER inicial (SQL Crudo)

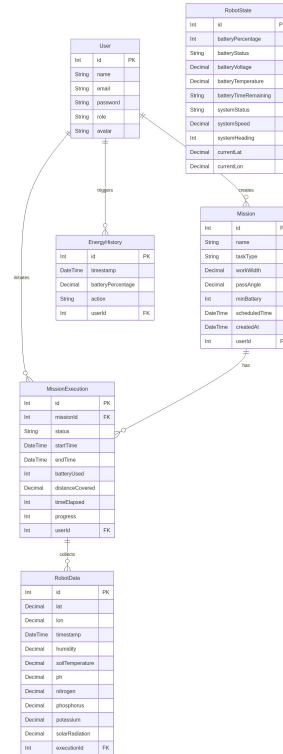


Figura 5.3: Diagrama ER autogenerado por Prisma

Integración de Sistemas y Simulación Telemétrica

Ante la cancelación definitiva de las pruebas con el chasis físico del robot, el desarrollo del servidor pivotó hacia la creación de un **simulador telemétrico avanzado**. Para soportar el envío masivo de estados robóticos, se sustituyeron las tradicionales peticiones HTTP orientadas a la gestión por una arquitectura de baja latencia impulsada por **WebSockets** bidireccionales. El servidor de AgroSkopos adoptó entonces el papel de despachador principal (*Broker*), capaz de inyectar variables cinemáticas, niveles de batería y coordenadas GPS sobre la red en milisegundos para actualizar la pantalla de los clientes en estricto tiempo real.

5.6. Desarrollo del Frontend

El cliente web evolucionó desde una sencilla interfaz de visualización estática hasta convertirse en una plataforma de gestión agrícola profundamente interactiva.

Cimentación Arquitectónica y Prototipado Visual

El desarrollo del *software* dio comienzo con la configuración del entorno moderno impulsado por **React** [22] y **Vite** [48]. Durante los primeros ciclos de desarrollo, se construyó el esqueleto de la *Single Page Application* (SPA), implementando el enrutamiento visual (React [22] Router) y maquetando el panel de control (*Dashboard*). El estado global se extrajo inmediatamente y se delegó a la librería **Zustand** [32], una decisión crítica en etapas tempranas que impidió que la complejidad estructural estrangulara el rendimiento al lidiar posteriormente con los flujos constantes de telemetría.

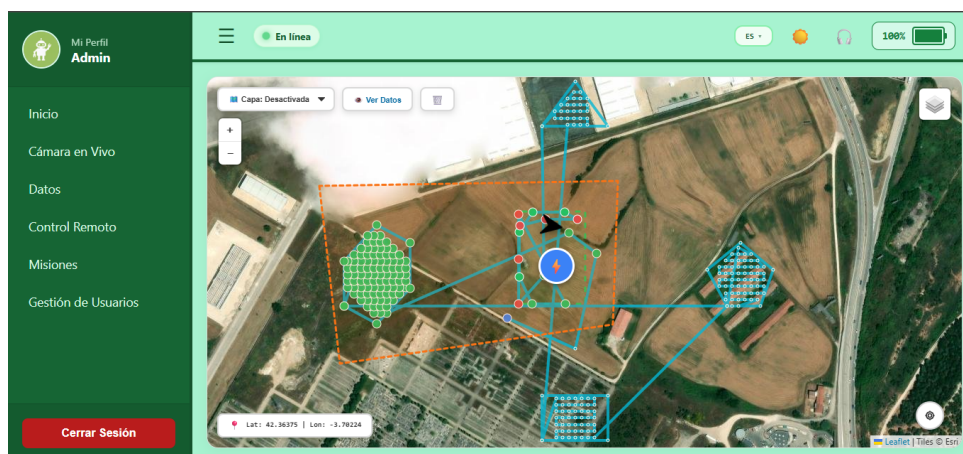


Figura 5.4: Panel principal de control (Dashboard) del sistema AgroSkopos

Control Cinemático y Analítica Espacial

Una vez construidos los cimientos, se integró el motor cartográfico **Leaflet** [1]. Se diseñaron interfaces responsivas para permitir el telemando manual direccional del vehículo a través de la web. Posteriormente, se programaron algoritmos de navegación automatizada para la generación visual de **rutas de cobertura** (como patrones agrícolas en *zigzag*).

A nivel analítico espacial, la aplicación introdujo dinámicas de *Geofencing* (delimitación poligonal de zonas seguras y parcelas) y la generación de

mapas de calor (*Heatmaps*) interactivos orientados a plasmar gráficamente la densidad paramétrica sobre el terreno, como las concentraciones de humedad y el nivel de pH edafológico. Además, el panel analítico es capaz de generar de forma nativa gráficas comparativas que cruzan el histórico de estos datos, permitiendo al operador visualizar la evolución temporal de los nutrientes y la batería directamente en la interfaz de la propia aplicación sin requerir herramientas externas.

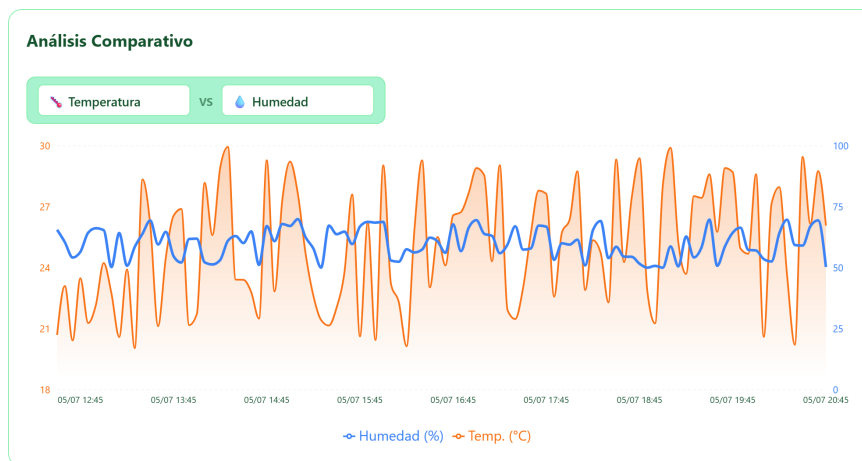


Figura 5.5: Gráfica comparativa de evolución temporal generada de forma nativa por la aplicación

Misiones Autónomas e Internacionalización

Con las lógicas de movimiento e interfaz estabilizadas, se diseñó el flujo operativo para dar soporte al ciclo completo de **Misiones Autónomas**. Los usuarios pueden planificar sobre un lienzo cartográfico, gestionar en tablas y poner en ejecución distintas misiones agrícolas que operan bajo una máquina de estados visual.

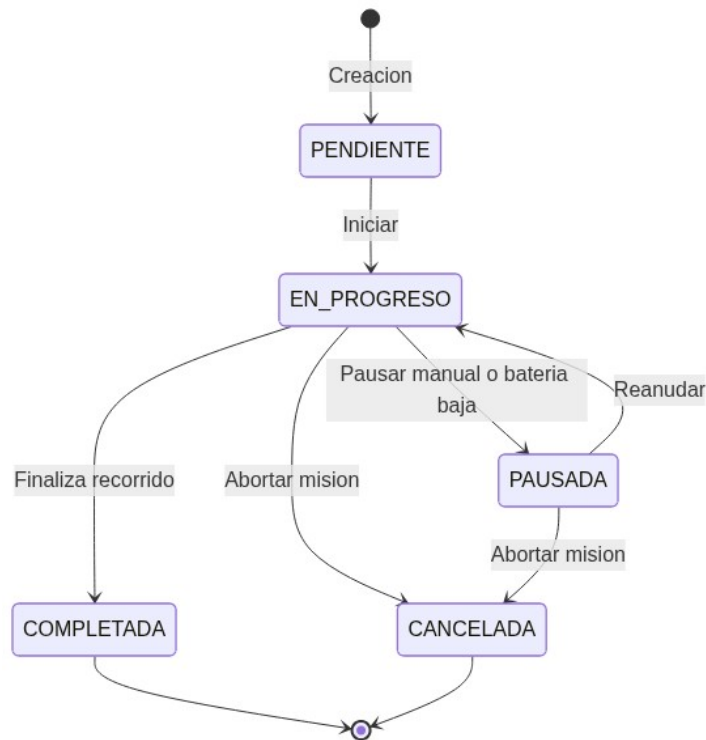


Figura 5.6: Máquina de estados del ciclo de vida de una Misión

Por último, se implementó la capa de soporte multi-idioma (i18n) a través del *framework* **i18next** [17], y se migró todo el cliente bajo los estándares de las **Aplicaciones Web Progresivas (PWA)**, habilitando el uso del portal como una aplicación nativa instalable capaz de cachear los mapas para ofrecer cargas casi instantáneas.

5.7. *Testing*, calidad de código y despliegue

Estrategia Integral de Pruebas

Para certificar la fiabilidad y robustez técnica de un sistema distribuido como AgroSkopos, se integró una cobertura analítica exhaustiva abordando cada capa crítica de su ecosistema:

1. **Pruebas Unitarias y de Componentes (*Frontend*):** Apoyándose en el corredor de pruebas **Vitest** [47] y en la biblioteca **React** [22] *Testing Library*, se verificó el correcto montaje y re-renderizado de la

interfaz. Se aislaron los componentes complejos utilizando simulaciones exhaustivas (*Mocks*) para validar la accesibilidad, las integraciones asíncronas de la API y las mutaciones persistentes en el gestor de estado global (Zustand [32]).

2. **Pruebas Unitarias lógicas (*Backend*):** En el seno del servidor, el *framework* **Jest** [21] permitió orquestar comprobaciones sobre los controladores y servicios lógicos de forma aséptica. Mediante el *mocking* inyectado, se desacoplaron las dependencias reales aislando la conexión a PostgreSQL [43] o a las pasarelas criptográficas para garantizar un rendimiento óptimo de las pruebas sin dependencias externas.
3. **Pruebas *End-to-End* (E2E) e Integración:** Representando el nivel superior de calidad en el servidor, se integró **Testcontainers** [3]. Esta herramienta orquesta contenedores efímeros de Docker [10] para instanciar bases de datos PostgreSQL [43] desechables exclusivas para el entorno de testeo. Esto aseguró que las peticiones simuladas por **Supertest** [19] interactuaran con un sistema vivo bajo condiciones idénticas al entorno productivo.
4. **Pruebas de Carga (*load testing*):** Para certificar que la arquitectura de eventos del servidor tolerara altos volúmenes de telemetría sin degradación, se diseñaron *scripts* asíncronos en **k6** [15]. Esta fase consistió en la inyección agresiva y parametrizada de cientos de usuarios virtuales concurrentes contra los *endpoints* críticos.

Para gobernar este proceso, los reportes estandarizados de cobertura de ambos clientes se fusionan mediante lógicas propias y son capturados automáticamente por un flujo de *GitHub Actions* [14]. Finalmente, dichos reportes nutren el **análisis estático de SonarCloud** [40], que actúa como **barrera técnica (*Quality Gate*)**, censurando y bloqueando el despliegue de cualquier confirmación que no alcance un mínimo del 80 % de cobertura o que introduzca agujeros de seguridad (*Security Hotspots*).

Como se puede apreciar en la figura 5.7, el panel de control de SonarCloud centraliza las métricas de calidad y confirma el paso exitoso de este riguroso *Quality Gate*. La imagen evidencia una **cobertura global superior al 90 %**, lo que significa que más de nueve de cada diez líneas de código de la aplicación están siendo evaluadas automáticamente en cada prueba. Este alto nivel de cobertura garantiza una extrema fiabilidad frente a errores (*bugs*), facilita enormemente la refactorización futura sin miedo a romper funcionalidades existentes y asegura la robustez del sistema en su conjunto. Además, el

panel muestra la ausencia total de vulnerabilidades y un bajísimo índice de deuda técnica (puntuación A en fiabilidad, seguridad y mantenibilidad), certificando la madurez del código base antes de su pase a los entornos de producción.

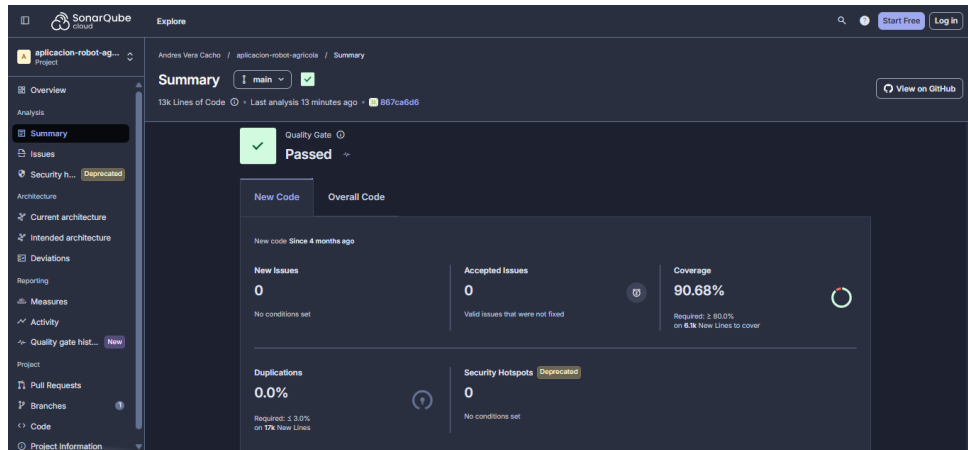


Figura 5.7: Panel principal de SonarCloud mostrando el estado del Quality Gate y la cobertura del proyecto

Despliegue y Arquitectura Cloud

Para garantizar una disponibilidad global y sin interrupciones, la plataforma completa de AgroSkopos fue orquestada y desplegada sobre un entorno productivo híbrido.

El sistema cliente (*Frontend*) se encuentra hospedado en **Vercel** [45]. Esta elección provee una **arquitectura de red de entrega de contenidos (CDN) global** altamente optimizada para construcciones de React [22], minimizando radicalmente la latencia para el operador agrícola independientemente de su localización.

Por el contrario, el procesamiento computacional pesado y el núcleo del servidor (*Backend*) residen en una **infraestructura virtualizada basada en Linux** desplegada a través de **Microsoft Azure** [23]. En este nodo central se implementó un flujo automatizado de orquestación local mediante **Docker** [10] **Compose**. Este archivo declarativo asegura el levantamiento secuencial y estandarizado del servidor Node.js [31], el **motor en memoria Redis** [35] (esencial para el gestor de colas en segundo plano) y el **clúster transaccional de PostgreSQL** [43].

La comunicación de este nodo interno con el mundo exterior fluye de manera exclusiva a través de un **proxy inverso gestionado por Nginx**. Además de balancear las rutas HTTP convencionales y la mensajería *Web-Socket*, este proxy absorbe la gestión y negociación de los **certificados criptográficos SSL/TLS**, garantizando que toda la telemetría operativa transite exclusivamente bajo el protocolo seguro **HTTPS**.

Trabajos relacionados

En el sector de la tecnología agrícola (*AgTech*), existen diversas soluciones comerciales y académicas orientadas a la monitorización y gestión de flotas. Sin embargo, la mayoría de estos sistemas se dividen en plataformas de muy alto coste orientadas a grandes latifundios (como las soluciones de maquinaria pesada) o en herramientas de recolección de datos satelitales que carecen de la capacidad de telemando en tiempo real.

A continuación, se describen brevemente algunas de las plataformas más representativas del sector y se establece una comparativa directa con las capacidades desarrolladas en AgroSkopos.

6.1. Soluciones Comerciales

John Deere Operations Center

Es una de las plataformas líderes a nivel global en la gestión agrícola. Permite la monitorización del rendimiento, la planificación de tareas y la integración con tractores automatizados. Si bien su ecosistema es sumamente robusto y maduro, se trata de una solución de código cerrado, altamente acoplada al *hardware* del propio fabricante (*vendor lock-in*) y cuyo coste de licenciamiento resulta prohibitivo para pequeños y medianos agricultores.

FarmLogs

FarmLogs es un *software* de gestión agrícola enfocado en el análisis de rentabilidad, seguimiento de lluvias y mapas satelitales. Aunque destaca en el procesamiento de datos a nivel macroscópico y planificación a largo plazo,

no está diseñado para el control de la telemetría en tiempo real ni posee capacidades de pilotaje remoto de vehículos autónomos.

Plataformas de Vuelo Agrícola (DJI SmartFarm / Pix4Dfields)

En el ámbito de la fotogrametría y la fumigación de precisión, destacan soluciones asociadas a drones como la serie DJI Agras operada mediante la aplicación DJI SmartFarm, o el *software* Pix4Dfields. Estas plataformas ofrecen una captura de datos aéreos incomparable y cuentan con sistemas de evitación física de obstáculos mediante radares y cámaras estéreo. Sin embargo, su operación está estrictamente limitada al vuelo y a ecosistemas cerrados, y no ofrecen la versatilidad de un *software* de control multipropósito de código abierto para robótica terrestre.

6.2. Comparativa Funcional

En la tabla 6.1 se sintetizan las principales diferencias funcionales entre las soluciones comerciales mencionadas y la propuesta implementada en este Trabajo de Fin de Grado. Para ofrecer una visión realista, se incluyen también aquellas características avanzadas propias del sector industrial que escapen al alcance actual de AgroSkopos.

Característica	John Deere	FarmLogs	DJI SmartFarm	AgroSkopos
Telemetría en Tiempo Real (Baja Latencia)	Sí	No	Sí	Sí
Telemando Manual (<i>WebSockets</i>)	No	No	No	Sí
Diseño de Misiones y Rutas Autónomas	Sí	No	Sí	Sí
Independencia de <i>Hardware</i> (Agnóstico)	No	Sí	No	Sí
Motor de Simulación Integrado	No	No	No	Sí
Gestión de Carga (Ej: Fumigación)	Sí	No	Sí	No
Evasión Física de Obstáculos (<i>Hardware</i>)	Sí	No	Sí	No
Detección Automática de Enfermedades (IA)	Sí	Parcial	Sí	No
Modelo de Licencia	Comercial	Comercial	Comercial	Código Abierto

Tabla 6.1: Comparativa de características entre plataformas

Como se observa en la comparativa, AgroSkopos logra un equilibrio único al ofrecer capacidades de telemetría y telemando de baja latencia en una plataforma agnóstica al *hardware* y de código abierto. Si bien carece de integraciones *hardware* profundas como la evasión física de obstáculos o la dosificación automatizada de líquidos (propias de soluciones industriales), sienta las bases lógicas necesarias para incorporarlas en el futuro.

Conclusiones y líneas de trabajo futuras

7.1. Conclusiones

El desarrollo de este Trabajo de Fin de Grado ha terminado de manera satisfactoria, logrando el cumplimiento de los objetivos propuestos desde su planificación. Con AgroSkopos, se ha materializado una plataforma web centralizada que da respuesta a las necesidades de monitorización, telemetría y control remoto en el ámbito de la agricultura de precisión. Mediante una arquitectura de baja latencia, se han satisfecho con éxito los requisitos funcionales exigidos: desde la implementación de un mapa interactivo y la gestión de misiones autónomas, hasta el despliegue de un panel de telemetría y control manual en tiempo real. Además, la adopción de metodologías ágiles y herramientas de análisis estático ha garantizado un producto final robusto, contenerizado y con altos estándares de calidad.

Desde la perspectiva de la ingeniería del *software*, el proyecto ha representado un valioso ejercicio de diseño arquitectónico y resolución de problemas complejos. El principal reto superado ha sido la construcción de una infraestructura escalable y de baja latencia, orquestando con éxito la **comunicación bidireccional en tiempo real** (WebSockets), la gestión eficiente del estado en el cliente y la automatización de pruebas mediante flujos de integración continua (CI/CD). Esta robustez estructural fue precisamente la que permitió una rápida adaptación técnica ante el retraso logístico del *hardware*.

En el plano formativo, este proyecto ha supuesto un cierre perfecto para mi formación en el grado. Asumir en solitario la responsabilidad integral del ciclo de vida del *software*, desde la toma de requisitos hasta las pruebas

automatizadas, ha permitido consolidar y poner en práctica de manera transversal los conocimientos teóricos adquiridos a lo largo de la carrera. Simultáneamente, el proyecto ha motivado un enriquecedor proceso de autoaprendizaje, fundamental para dominar y aplicar tecnologías modernas ampliamente demandadas en el sector.

En definitiva, AgroSkopos se consolida como un ecosistema tecnológico integral orientado a la digitalización del sector agrario. Su evolución desde un prototipo conceptual hasta una plataforma *cloud-ready*, asíncrona y sometida a rigurosas pruebas de carga, demuestra la viabilidad de aplicar arquitecturas de *software* modernas (propias de sistemas de alta transaccionalidad) al control en tiempo real de maquinaria pesada.

7.2. Líneas de trabajo futuras

El desarrollo y la posterior evaluación de AgroSkopos han puesto de manifiesto la viabilidad de la plataforma como núcleo tecnológico operativo, abriendo a su vez nuevas e interesantes vías de investigación e ingeniería para enriquecer el sistema en futuras iteraciones.

Transición a *Hardware* Real y Comunicaciones Avanzadas

El sistema actual se apoya de forma robusta en un simulador nativo diseñado para replicar la dinámica del entorno real. El siguiente paso natural en la evolución del proyecto es sustituir este motor de *software* por una **pasarela IoT** (*Internet of Things*) que establezca comunicación directa con el chasis de un robot físico real. Para lograrlo, se propone el uso del estándar *Robot Operating System* (ROS2) integrado con MQTT, lo que garantizaría un flujo telemétrico eficiente.

Además, actualmente el visor de cámara emplea una simulación funcional integrada en la interfaz; la mejora propuesta consiste en implementar un servidor de *streaming* nativo basado en la tecnología **WebRTC**. Esto permitirá transmitir el flujo de vídeo en vivo directamente desde las cámaras del robot hacia el panel de control del operador con latencias sub-segundo, un factor crítico e indispensable para asegurar la maniobrabilidad durante la operación manual a distancia.

Por otra parte, se plantea adaptar la arquitectura del sistema para soportar **operaciones sin conexión** en entornos agrícolas caracterizados por conectividad intermitente. La solución pasaría por dotar al cliente de la

capacidad de almacenar localmente tanto la telemetría generada como el registro temporal de misiones y captura de datos. Al recuperar la señal de red, el sistema orquestará una sincronización diferida automática contra la base de datos central en PostgreSQL [43], asegurando la integridad transaccional de los datos agrícolas sin riesgo de pérdida.

Visión Artificial, Realidad Aumentada y Registro Visual

Aprovechando el flujo continuo de vídeo en tiempo real implementado en el control remoto, el ecosistema puede beneficiarse enormemente de la integración de modelos de **Machine Learning** (tales como la arquitectura YOLOv8). Al procesar las imágenes en el propio extremo de la red (**Edge Computing**), la inteligencia artificial sería capaz de identificar amenazas agronómicas, como la presencia de malas hierbas o indicadores visuales de enfermedades en las plantaciones.

Mediante el uso de librerías nativas del navegador web como WebGL o Canvas API, la información detectada por la IA se proyectaría directamente sobre el vídeo simulando una interfaz de tipo HUD (*Head-Up Display*). Este enfoque enriquecería sustancialmente la experiencia del operador, permitiéndole no solo observar el terreno a través de la cámara, sino también visualizar marcadores geográficos precisos, trazados dinámicos de rutas, delimitaciones de geocercas y alertas tempranas, multiplicando la conciencia situacional en tiempo real.

Complementariamente, el sistema integraría un módulo de captura y almacenamiento geolocalizado de imágenes. Esto facilitaría el trabajo de ingeniería agrícola al permitir la configuración autónoma del robot para registrar el estado visual del cultivo fotograma a fotograma en los distintos puntos de recolección de datos (*waypoints*), generando en la base de datos un inventario y un rastro visual persistente asociado a las coordenadas GPS exactas.

Planificación Óptima y Control Cinemático Avanzado

Aunque los algoritmos de navegación actuales garantizan una cobertura de área funcional (mediante patrones estáticos en *zigzag*), existe un gran margen de mejora hacia la planificación adaptativa inteligente. El sistema evolucionaría con la introducción de **algoritmos de búsqueda heurística de grafos**, tales como A* o D* Lite, los cuales se aplicarían no sobre un plano liso, sino sobre un sofisticado "mapa de costes" topográfico. La IA evaluaría

en tiempo real diversas condiciones críticas del entorno (desde la orografía y el ángulo de inclinación del terreno, hasta la densidad del suelo) para trazar la ruta óptima de menor esfuerzo cinemático, reduciendo drásticamente tanto el tiempo empleado por misión como el consumo energético total.

En escenarios reales de operación, la seguridad del robot frente a imprevistos es ineludible. Por este motivo, la plataforma debe integrar en sus algoritmos el procesamiento en bruto procedente de escáneres LIDAR y sensores de proximidad ultrasonidos. La aplicación de técnicas de **evasión dinámica de obstáculos** (como los Histogramas de Campo Vectorial) facultaría al robot agrícola para sortear automáticamente maquinaria u operarios que interfirieran accidentalmente en su trayectoria predefinida, garantizando una maniobra segura y la posterior corrección automatizada hacia el objetivo original.

Escalabilidad hacia la Gestión de Flotas (*Swarm Robotics*)

AgroSkopos, en su versión fundacional, ha optimizado su capa de servicios y comunicación de eventos asumiendo un escenario operativo de un único vehículo agrícola. No obstante, la robustez de su motor de bases de datos PostgreSQL [43] y su canal de WebSockets soportan teóricamente arquitecturas de mayor densidad y demanda de flujo. La proyección hacia el futuro de esta herramienta radicaría en rediseñar la plataforma de control como un **administrador de flotas centralizado**, dotando a la interfaz de las capacidades de monitorización múltiple y en paralelo. Esta evolución orquestaría escenarios de cooperación avanzada, posibilitando a un único operador humano la supervisión general y la delegación de tareas concurrentes a múltiples robots agrícolas (*enjambres*) operando en sincronía a lo largo de extensiones de cultivo a gran escala.

Migración a Tipado Estricto (TypeScript)

Por último, para dotar al producto de un estándar de mantenibilidad superior y facilitar la incorporación futura de nuevos desarrolladores al equipo, el proceso lógico siguiente consistiría en la migración progresiva del código fuente (actualmente escrito en JavaScript) hacia **TypeScript**. Adoptar un sistema de tipado estricto en el compilador permitiría detectar errores semánticos y de interfaz en etapas tempranas del desarrollo, ofreciendo un contrato robusto de tipos de datos, especialmente crítico cuando se manejan estructuras telemétricas tan complejas y densas provenientes del robot.

Bibliografía

- [1] Vladimir Agafonkin. Leaflet: an open-source javascript library for mobile-friendly interactive maps. <https://leafletjs.com/>, 2026.
- [2] Andris Reinman. Nodemailer: Send e-mails from node.js. <https://nodemailer.com/>, 2026.
- [3] AtomicJar, Inc. Testcontainers. <https://testcontainers.com/>, 2026.
- [4] Auth0. Json web tokens. <https://jwt.io/>, 2026.
- [5] Axios. Axios: Promise based http client for the browser and node.js. <https://axios-http.com/>, 2026.
- [6] Bcrypt. Bcrypt password hashing function. <https://en.wikipedia.org/wiki/Bcrypt>, 2026.
- [7] Nyle C Brady and Ray R Weil. *The nature and properties of soils*, volume 14. Prentice Hall Upper Saddle River, NJ, 2008.
- [8] Colin McDonnell. Zod: Typescript-first schema validation with static type inference. <https://zod.dev/>, 2026.
- [9] Digital Science. Overleaf: Collaborative latex editor. <https://www.overleaf.com/>, 2026.
- [10] Docker Inc. Docker: Accelerated container application development. <https://www.docker.com/>, 2026.
- [11] European Space Agency. Telemetry, tracking and command. <https://www.esa.int/>, 2026.

- [12] F5, Inc. Nginx: High performance load balancer, web server, & reverse proxy. <https://www.nginx.com/>, 2026.
- [13] Food and Agriculture Organization. Fertilizer and plant nutrition bulletin. <https://www.fao.org/>, 2026.
- [14] GitHub, Inc. Github actions. <https://github.com/features/actions>, 2026.
- [15] Grafana Labs. k6: Open source load testing tool. <https://k6.io/>, 2026.
- [16] Helmet.js. Helmet: Secure express apps by setting various http headers. <https://helmetjs.github.io/>, 2026.
- [17] i18next. i18next: internationalization-framework. <https://www.i18next.com/>, 2026.
- [18] James Hall. jspdf: Client-side javascript pdf generation. <https://parall.ax/products/jspdf>, 2026.
- [19] T. Lad. Supertest. <https://github.com/ladjs/supertest>, 2026.
- [20] Leslie Lamport. Latex: A document preparation system. <https://www.latex-project.org/>, 1985.
- [21] Meta Platforms, Inc. Jest: Delightful javascript testing. <https://jestjs.io/>, 2026.
- [22] Meta Platforms, Inc. React: The library for web and native user interfaces. <https://react.dev/>, 2026.
- [23] Microsoft Corporation. Microsoft azure: Cloud computing services. <https://azure.microsoft.com/>, 2026.
- [24] Microsoft Corporation. Visual studio code. <https://code.visualstudio.com/>, 2026.
- [25] Hossein Mousazadeh. A technical review on navigation systems of agricultural tractors. *Journal of Terramechanics*, 105:73–93, 2013.
- [26] David J Mulla. Twenty five years of remote sensing in precision agriculture: Key advances and remaining knowledge gaps. *Biosystems engineering*, 114(4):358–371, 2013.

- [27] Nathan Friedly. express-rate-limit: Basic rate-limiting middleware for express. <https://github.com/express-rate-limit/express-rate-limit>, 2026.
- [28] Niklas von Herten. html2canvas: Screenshots with javascript. <https://html2canvas.herten.com/>, 2026.
- [29] OpenJS Foundation. Eslint - pluggable javascript linter. <https://eslint.org/>, 2026.
- [30] OpenJS Foundation. Express - node.js web application framework. <https://expressjs.com/>, 2026.
- [31] OpenJS Foundation. Node.js. <https://nodejs.org/>, 2026.
- [32] Poimandres. Zustand: Bear necessities for state management in react. <https://zustand-demo.pmnd.rs/>, 2026.
- [33] Prisma Data, Inc. Prisma: Next-generation node.js and typescript orm. <https://www.prisma.io/>, 2026.
- [34] Recharts. Recharts: A composable charting library built on react components. <https://recharts.org/>, 2026.
- [35] Redis Ltd. Redis: The open source, in-memory data store. <https://redis.io/>, 2026.
- [36] Remix Software. React router. <https://reactrouter.com/>, 2026.
- [37] Sasha Koss. date-fns: Modern javascript date utility library. <https://date-fns.org/>, 2026.
- [38] Ken Schwaber and Jeff Sutherland. The scrum guide. <https://scrumguides.org/>, 2020.
- [39] Socket.IO. Socket.io. <https://socket.io/>, 2026.
- [40] SonarSource SA. Sonarqube / sonarcloud. <https://www.sonarsource.com/>, 2026.
- [41] Taskforce.sh Inc. Bullmq: Premium message queue for node.js. <https://docs.bullmq.io/>, 2026.
- [42] Testing Library. React testing library. <https://testing-library.com/docs/react-testing-library/intro/>, 2026.

- [43] The PostgreSQL Global Development Group. Postgresql: The world's most advanced open source relational database. <https://www.postgresql.org/>, 2026.
- [44] Turf.js. Turf.js - advanced geospatial analysis for browsers and node.js. <https://turfjs.org/>, 2026.
- [45] Vercel Inc. Vercel: Develop. preview. ship. <https://vercel.com/>, 2026.
- [46] Vite PWA. Vite plugin pwa. <https://vite-pwa-org.netlify.app/>, 2026.
- [47] Vitest Contributors. Vitest: A vite-native testing framework. <https://vitest.dev/>, 2026.
- [48] Evan You. Vite: Next generation frontend tooling. <https://vitejs.dev/>, 2026.
- [49] James Yu. Latex workshop extension for visual studio code. <https://marketplace.visualstudio.com/items?itemName=James-Yu.latex-workshop>, 2026.
- [50] Naiqian Zhang, Maohua Wang, and Ning Wang. Precision agriculture—a worldwide overview. *Computers and electronics in agriculture*, 36(2-3):113–132, 2002.
- [51] Zube. Zube: Agile project management for github. <https://zube.io/>, 2026.